任务空间速度控制代码说明

基于公式236的6DOF扩展实现

OSIC Force Control Project

2026 年 3 月 24 日

1 控制参数设置

Listing 1: Control parameters

```
def setup_control_parameters(self):
    """Set control parameters (Eq.236)"""
    self.K_p = np.diag([5.0, 5.0, 5.0, 3.0, 3.0, 3.0])
```

说明：

- 前3个元素对应旋转控制
- 后3个元素对应位置控制

2 Jacobian计算

Listing 2: Jacobian computation

```
def get_jacobian_6x7(self):
    """Get 6DOF Jacobian"""
    jacp = np.zeros((3, self.model.nv))
    jacr = np.zeros((3, self.model.nv))

    hand_body_id = self.model.body("panda_hand").id
    mujoco.mj_jacBody(self.model, self.data, jacp, jacr,
        hand_body_id)

    J = np.zeros((6, self.n_joints))
    for i in range(self.n_joints):
        dof_idx = self.joint_dof_indices[i]
```

```
        if dof_idx >= 0:
            J[:3, i] = jacr[:, dof_idx]
            J[3:, i] = jacp[:, dof_idx]

    return J
```

说明：

- 前3行为角速度
- 后3行为线速度

3 配置误差计算

Listing 3: Configuration error

```
def compute_configuration_error(self, pos_curr, quat_curr,
    pos_ref, quat_ref):
    """Compute X_e = log(X^{-1} X_d)"""

    rot_curr = Rotation.from_quat(quat_curr)
    rot_ref = Rotation.from_quat(quat_ref)

    R_curr = rot_curr.as_matrix()
    R_ref = rot_ref.as_matrix()

    T_curr = np.eye(4)
    T_curr[:3, :3] = R_curr
    T_curr[:3, 3] = pos_curr

    T_ref = np.eye(4)
    T_ref[:3, :3] = R_ref
    T_ref[:3, 3] = pos_ref

    T_curr_inv = np.eye(4)
    T_curr_inv[:3, :3] = R_curr.T
    T_curr_inv[:3, 3] = -R_curr.T @ pos_curr

    T_err = T_curr_inv @ T_ref

    X_e = self.se3_log_map(T_err[:3, :3], T_err[:3, 3])
```

```
    return X_e
```

4 控制律

Listing 4: Control step

```
def control_step(self, t, dt=0.002):
    """Task-space velocity control"""

    pos_curr = np.array(self.data.body("panda_hand").xpos)
    quat_curr = np.array(self.data.body("panda_hand").xquat)

    J = self.get_jacobian_6x7()

    z_des = 0.4 - min(t / 2.0, 1.0) * 0.25
    pos_ref = np.array([0.4, 0.0, z_des])

    vel_ref_linear = np.array([0.0, 0.0, -0.125 if t < 2.0 else
        0.0])
    vel_ref_angular = np.zeros(3)

    vel_ref_6d = np.concatenate([vel_ref_angular, vel_ref_linear
        ])

    X_e = self.compute_configuration_error(
        pos_curr, quat_curr, pos_ref, quat_curr
    )

    X_dot = vel_ref_6d + self.K_p @ X_e

    J_pinv = np.linalg.pinv(J, rcond=1e-6)
    qdot = J_pinv @ X_dot

    qdot_max = np.array([2.0,2.0,2.0,2.0,2.5,2.5,2.5])
    qdot = np.clip(qdot, -qdot_max, qdot_max)

    for i, name in enumerate(self.velocity_actuator_names):
        ctrl_id = self.model.actuator(name).id
        self.data.actuator(ctrl_id).ctrl = float(qdot[i])
```